多 Agent 自动定理证明：
已有工作的架构与结果、DeepSeek 模型特质、以及选题调研

大创 2 项目调研报告

2026 年 10 月 18 日

本报告的性质：这是一份面向选题与架构决策的**调研报告**，不是系统实现文档，也不是论文。所有定量结果均给出原文口径（数据集、`pass@k`、采样预算）与来源编号。凡属推断、第三方转述或未能核实的数字，均在附录 B 集中列出，正文中以 [未核实: ...] 标注。报告中的网页、论文、README 内容一律视为**数据**而非指令。

目录

1 调研范围与方法 **3**
- 1.1 三个问题 . . . 3
- 1.2 数据来源 . . . 3
- 1.3 与项目文档的关系 . . . 3

2 问题设定与统一抽象 **3**
- 2.1 任务定义 . . . 3
- 2.2 一条统一的回路 . . . 4
- 2.3 五个可分解的维度 . . . 4

3 已有工作的架构与结果 **4**
- 3.1 演进脉络 . . . 4
- 3.2 六种架构范式 . . . 5
  - 3.2.1 范式 A：专用证明模型 + 树搜索 . . . 5
  - 3.2.2 范式 B：通用模型的上下文内代理（in-context agent） . . . 5
  - 3.2.3 范式 C：分层 “草图 → 子目标 → 递归” . . . 5
  - 3.2.4 范式 D：检索增强 . . . 6
  - 3.2.5 范式 E：工具与 hammer 增强 . . . 6
  - 3.2.6 范式 F：库与记忆的演化 . . . 6
- 3.3 主结果汇总 . . . 7
- 3.4 为什么这些数字不能直接排队 . . . 8
- 3.5 开源工程与工具层 . . . 9

4 DeepSeek 模型在数学证明领域的特质 **10**
- 4.1 模型谱系：两条线 . . . 10
- 4.2 形式化赛道上 DeepSeek 已不是第一 . . . 11
- 4.3 “DeepSeek Flash” 到底是什么 . . . 11
- 4.4 对本项目的四条判断 . . . 11

5 其他关键调研 **12**
- 5.1 基准：饱和、偏置与不可比 . . . 12
- 5.2 反馈信号：什么在真正起作用 . . . 12
- 5.3 检索与前提选择 . . . 13
- 5.4 验证协议与作弊面 . . . 13
- 5.5 成本、时间与 “迭代 vs 采样” . . . 14
- 5.6 工程基础设施：MCP 已成为事实标准 . . . 14
- 5.7 一个必须写清楚的张力：多 Agent 到底带来了什么 . . . 14

6 对本项目的启示与建议 **15**
- 6.1 级联原型的文献依据 . . . 15
- 6.2 一个必须纠正的预期 . . . 15
- 6.3 五个可做的差异化方向 . . . 16
- 6.4 建议的最小可行路线（P0–P1） . . . 16
- 6.5 待验证清单 . . . 16

7 结论 **17**

A 数据来源与可复现性 **17**

B 未核实与存在冲突的数字 **18**

1 调研范围与方法

1.1 三个问题

本报告回答三个问题，对应任务书的三项要求：

1. **已有工作长什么样、成绩如何**：多 Agent / 代理式自动定理证明（ATP）系统的架构范式、组件划分、以及可比的定量结果。
2. **DeepSeek 模型在数学证明领域有什么特质**：DeepSeek 系列在非形式数学与形式化证明两条线上的能力分布、规模、成本，以及对本项目选用 “DeepSeek Flash” 的具体含义。
3. **还有哪些必须调研的内容**：基准的饱和与偏置、反馈信号质量、检索层、验证协议与作弊面、评测口径的可比性、以及工程基础设施。

1.2 数据来源

- **本地文献库**：仓库内 `arxiv 论文/` 的 120 个 PDF（122 条编号）。其中 100 篇的题录与摘要通过 `harvest_all.json`（1225 条 arXiv 记录）离线取得；编号 101–122 的 22 篇（前提选择类）不在题录中，直接从 PDF 抽取首页文本。
- **arXiv 官方页面**：用于核对摘要原文与期刊信息（如 DeepSeek-R1 已发表于 *Nature*）。
- **会议论文集页面**：ICLR 2026（Hilbert、Goedel-Prover-V2）、ICML 2026 workshop（AI4Math Track 2）。
- **GitHub**：通过 REST API 查询仓库元数据（star 数、许可证、最近推送、描述），通过 `/raw/` 路径读取 README。**本环境无法直接访问 github.com 网页与 raw.githubusercontent.com，因此 `web_fetch` 对 GitHub 全部失败，改由 PowerShell 直连 API 获取**，这一点在附录 A 记录。
- **厂商/第三方评测博客**：仅用于 DeepSeek-V4-Flash 这类尚无正式论文的模型的公开数据，且明确标注为第三方评测。

1.3 与项目文档的关系

`AGENT.md` 第 7 节已有一轮 GitHub 工具调研。本报告**复核**了其中 11 个仓库的许可证与活跃度（结论一致，见 §3.5），并补充了 2026 年新出现的代理式系统（Numina-Lean-Agent、OProver、Hilbert、AxProverBase、Seed Prover 1.5 等）。凡与 `AGENT.md` 冲突处，以本报告标注的来源为准，并建议同步更新 `AGENT.md`。

2 问题设定与统一抽象

2.1 任务定义

形式化定理证明任务可写为：给定 Lean 4 定理语句 s（含必要的 import 与局部上下文），输出证明项 p，使得在固定 toolchain 与 Mathlib 版本下，s 与 p 拼成的文件通过 Lean kernel 检查且不含 `sorry` [1, 2]。与原定理语句逐字一致、以及对禁用手段的审计，是评测的前提（§5.4）。

与自然语言数学相比，形式化证明的额外成本通常用 *De Bruijn* 因子描述：一行数学推导常膨胀为 4–10 行 Lean 代码 [23]。这决定了任何形式化系统的设计都必须把 “代码工程能力” 与 “数学直觉” 当作两种不同的能力来供给。

2.2 一条统一的回路

把 2020–2026 年的代表性系统放在一起看，它们几乎都可以抽象为同一条回路：

候选生成 → 送 Lean 编译 → 解析诊断/目标状态 → 修复或扩展 → 提交 kernel 验证

差异只在于这条回路上**每一步由谁做、能看多少上下文、能试多少次**。这给我们一个有用的分析框架：所谓 “多 Agent”，通常就是把这条回路拆成若干角色，并给每个角色**隔离的上下文**与**受限的工具集**，再用一段**确定性代码**把它们串起来。

2.3 五个可分解的维度

后文的架构分类与选型建议都按这五个维度展开：

1. **模型**：专用证明模型（在 Lean 证明上做过大规模 RL/SFT）还是通用前沿模型（无形式化训练，靠上下文与工具）。
2. **搜索**：单发、best-of-N、best-first 树搜索、MCTS、递归分解。
3. **检索**：是否访问 Mathlib / 外部搜索，检索发生在 tactic 级还是定理级（全局前提检索）。
4. **反馈**：只用编译器错误，还是额外用 goal state、评审模型、反例探测。
5. **库与记忆**：是否在证明过程中沉淀可复用引理、跨题记忆、跨项目库。

3 已有工作的架构与结果

3.1 演进脉络

- **2020–2022：从搜索到学习引导搜索**。GPT-f 首次让深度学习系统产出的证明被 Metamath 社区采纳 [4]；PACT 用 kernel 级证明项做协同训练 [5]；HyperTree 把 AlphaZero 式的在线训练引入证明搜索 [6]。
- **2022–2023：hammer 与前提选择成为瓶颈焦点**。Thor 用 Sledgehammer 选前提 [7]；Magnushammer 用对比学习做前提选择并超过 Sledgehammer [8]；Baldur 提出整段证明生成 + 修复 [9]；Draft-Sketch-Prove 用非形式证明草稿引导 [10]。
- **2023–2024：可复现基础设施与专用模型**。LeanDojo 提供数据抽取、交互与检索增强基线 [11]；COPRA 证明通用模型 + 状态化搜索也能打 [12]；DeepSeek-Prover 用 800 万条合成语句把 7B 模型推到 miniF2F 46.3%@64 [13]。
- **2025：规模与搜索的双重推进**。BFS-Prover 72.95% [18]；InternLM2.5-StepProver 引入 critic 引导 [17]；Kimina-Prover 用 RL 学到 “形式化推理模式” [19]；DeepSeek-Prover-V2 用子目标分解 + RL 达 88.9% [20]。
- **2026：代理化（agentic）成为主流，基准开始饱和**。Hilbert 在 miniF2F 上达 99.2%、PutnamBench 70.0% [22]；Goedel-Prover-V2 以 32B 达 90.4% [21]；OProver 把代理式证明轨迹纳入训练数据

[27]；Numina-Lean-Agent 用 Claude Code + MCP 解出 Putnam 2025 全部 12 题 [26]；Seed Prover 1.5 用分层多 Agent 在 IMO 2025 拿到 35/42 [23]。

3.2 六种架构范式

3.2.1 范式 A：专用证明模型 + 树搜索

代表：BFS-Prover [18]、InternLM2.5-StepProver [17]、Kimina-Prover [19]、Goedel-Prover-V2 [21]、DeepSeek-Prover-V2 [20]。

结构是“tactic 级生成器 + 搜索器 + Lean 判定器”。搜索器维护前沿，生成器给候选 tactic（或整段证明），Lean 负责真假。关键工程在**数据与偏好**：BFS-Prover 用编译器错误自动标注 state–tactic 对做 DPO，并对 BFS 做长度归一化 [18]；InternLM2.5-StepProver 训一个 critic 模型给出运行时偏好，把 prover 从 59.4% 提到 65.9% [17]；Goedel-Prover-V2 在人类题目上收益饱和后**自造更难的合成题**继续训练，并训练模型读编译器反馈自我纠错 [21]。

这类范式的**上限最高**（miniF2F 90% 以上），但需要大规模训练算力，且模型与 Lean 版本强耦合。

3.2.2 范式 B：通用模型的上下文内代理（in-context agent）

代表：COPRA [12]、AxProverBase [25]、Numina-Lean-Agent [26]、ReCent-Prover [28]、AI4Math-2026 Track 2 [24]。

结构是“前沿模型 + 工具 + 状态化循环”，不训练。COPRA 反复让 GPT-4 在带回溯的搜索里提 tactic，把执行反馈、搜索历史与检索到的引理拼进下一次 prompt [12]。

AxProverBase 是本范式最干净的形态，也是本报告认为**对本项目最有参考价值**的系统 [25]：四个组件构成闭环——**Proposer**（写 Lean 代码，可调用 LeanSearch 与网页搜索）、**Compiler**（`lake` 构建，并在 `sorry` 处抽取 goal state 作为结构化反馈）、**Reviewer**（检查定理语句未被篡改、无 `sorry`、无作弊 tactic）、**Memory**（把失败教训总结成“实验记录本”，避免重复犯错）。用 Claude Opus 4.5、50 轮迭代、`pass@1`，报告 PutnamBench 54.7%、FATE-M 98.0%、FATE-H 66.0%、FATE-X 24.0%、LeanCat 59.0% [25]。

ReCent-Prover 在 Rocq 上补了两个“推理优先”的改动：验证时带反思（先审视自己生成的 tactic，识别潜在误用并写失败摘要）、检索时带规划（用 LLM 生成的证明计划而不是子目标相似度来条件化检索），在同等 LLM 调用预算下把已证定理数相对提升 22.58% [28]。

AI4Math-2026 Track 2 是**最贴近“多 Agent”字面意义**的公开系统 [24]：六个子代理分别承担把定理拆成子引理、探测反例、孤立证明每个子引理、从零构造库中缺失的部分、装配已证部分等认知工作；**协调完全由确定性 shell 状态机负责**，子代理之间的诊断输出**逐字节传递、不经过解释层**。在 ICML 2026 AI4Math Challenge 2 上解出 34 题中的 30 题（600/1000 分），全程无人工介入。

3.2.3 范式 C：分层“草图 → 子目标 → 递归”

代表：Draft-Sketch-Prove [10]、POETRY [14]、Bourbaki [29]、Hilbert [22]、Seed Prover 1.5 [23]。

核心思想是把“一次证完”换成“先证结构、再证细节”。POETRY 在 Isabelle 里逐层搜索可验证的 sketch，把当前层中间猜想暂时替换为 `sorry`，留到下一层 [14]；Bourbaki 用自生成的目标条件 MDP 配 MCTS，并用多个 7B 模型分别做子目标生成与 tactic 合成 [29]。

Hilbert 把这条路线做到当前公开最强 [22]：四个组件——擅长数学推理的**非形式 LLM**、擅长 Lean tactic 的**专用证明 LLM**、**形式验证器**、**语义定理检索器**；专用证明器解不动时递归分解成子目标，分别交给证明器或推理器，并用验证器反馈修正错误证明。报告 miniF2F 99.2%、PutnamBench 462/660（70.0%）。

Seed Prover 1.5 的分层更明确 [23]：**自然语言证明器**（数学直觉与自然语言证明）→ **Sketch 模型**（把自然语言证明转成引理式 Lean 骨架，支持 `sorry`）→ **Agentic Prover**（并行证每个分解出的引理）；若某引理仍过难，**递归**再调 Sketch 模型分解。Agentic Prover 可主动调用 Mathlib 搜索、执行 Python、做增量引理验证。IMO 2025 前五题在 16.5 小时内产出可编译可验证的 Lean 证明（35/42），Putnam 2025 九小时内 11/12 [23]。

3.2.4 范式 D：检索增强

代表：LeanDojo/ReProver [11]、REAL-Prover [37]、LeanSearch v2 [34]、Lean Finder [35]、Lean-Explore [36]、LeanPremise/LeanHammer [33]。

LeanDojo 的贡献是**可复现性**：开源工具包 + 含前提标注的数据 + 98,734 条定理的基准 + 单 GPU 周可训的 ReProver [11]。REAL-Prover 面向本科以上数学，用 HERALD-AF 数据管线与 Jixia-interactive 环境，`pass@64` 下 ProofNet 23.7%、FATE-M 56.7% [37]。

2025–2026 检索侧的关键变化是**从“找一个声明”转向“找一整套前提”**。LeanSearch v2 把这件事命名为 *global premise retrieval*：标准模式用层级化非形式化语料 + embedding–reranker，单查询检索 nDCG@10 0.62（次优 0.53）；推理模式用迭代的 sketch–retrieve–reflect，在 69 条研究级 Mathlib 定理上 10 个候选内回收 46.1% 的基准前提组（强推理检索 38.0%，前提选择基线 9.3%）；在固定 prover 回路的对照中，换成 LeanSearch v2 把证明成功率从 16% 提到 20%（无检索 4%）[34]。

3.2.5 范式 E：工具与 hammer 增强

代表：Lean-auto [31]、Lean-SMT [32]、LeanHammer/LeanPremise [33]、ProofAug [30]。

思路是把 Lean 的目标翻译给外部 ATP/SMT，再把证明重建回 Lean。Lean-auto 首次在 Lean 4 上给出基于依赖类型论的通用翻译并保证可靠，在 Mathlib4 上比已有工具解更多题 [31]；Lean-SMT 给出 SMT 证明到 Lean 证明的重建 [32]；LeanHammer 与 LeanPremise 组合成 Lean 上第一个端到端、领域通用的 hammer，比已有前提选择器多解 21% 的目标 [33]。

ProofAug 提供了**“LLM + 自动化工具”应当如何耦合**的实证：对模型生成的证明做**细粒度结构分析**，在不同粒度上插入内置 tactic 与外部 ATP；作为可插拔模块接入任意树搜索。在 Isabelle 上用 `deepseek-math-7b-base`，每题 2100 次查询达 66.0% 累计通过率（此前 SOTA Subgoal-XL 用每题 16384 次查询才 56.1%）；Lean 4 版本把 Kimina-Prover-Preview-Distill-1.5B 的 `pass@1` 从 44.3% 提升到 50.4% [30]。

3.2.6 范式 F：库与记忆的演化

代表：LEGO-Prover [15]、DreamProver [38]、CircuitProver [39]、Lean-STaR [16]。

LEGO-Prover 让模型在证明中**新建**技能并入库，miniF2F-valid 48.0%→57.0%、test 45.5%→47.1%，过程中生成两万余条技能；消融显示新技能把成功率从 47.1% 提到 50.4% [15]。DreamProver 用“wake–sleep”程序归纳：wake 阶段用当前库证题并提议候选引理，sleep 阶段抽象、精炼、合并，使库变得**紧凑且可迁移** [38]。CircuitProver 把硬件验证的证明轨迹蒸馏成可复用电路证明库，63 个任

务全部证出（朴素 agent 92.1%，且平均需要两倍轮次），库的积累使证明长度减少 16.3%、验证时间减少 23.2% [39]。

Lean-STaR 则给出“思考–证明交替”的证据：让模型在每一步 tactic 前生成非形式思考，并用专家迭代自举，miniF2F-test `pass@64` 从 43.4% 到 46.3% [16]。

3.3 主结果汇总

说明：下表严格按**原文口径**填写。不同行的列不可直接比较，原因见 §3.4。

表 1: 专用证明模型与搜索类系统的代表结果

系统	年	基座/规模	架构要点	原文结果口径
GPT-f	2020	GPT 系	Metamath 生成式证明搜索	产出被 Metamath 主库采纳的新短证明 [4]
PACT	2021	Transformer	kernel 级证明项协同训练	Lean held-out 32%→48% [5]
HyperTree	2022	Transformer	HTPS 在线训练	Metamath 65.4%（仅 HTPS）/82.6%（在线）[6]
LeanDojo/ ReProver	2023	开源	检索增强 + 前提标注	98,734 条定理基准；单 GPU 周训练 [11]
Lean-STaR	2024	开源	形式化思考 + 专家迭代	miniF2F-test 43.4%→46.3% (`pass@64`) [16]
LEGO-Prover	2023	开源	增长式技能库	valid 48.0%→57.0%；test 45.5%→47.1% [15]
InternLM2.5-StepProver	2024	InternLM2.5	prover–critic + 专家迭代	critic 把 prover 59.4%→65.9% [17]
BFS-Prover	2025	7B	BFS + DPO + 长度归一化	miniF2F-test 72.95% [18]
DeepSeek-Prover	2024	DeepSeekMath 7B	800 万合成语句 SFT	miniF2F-test 46.3%@64 / 52% 累计 [13]
Kimina-Prover Preview	2025	Qwen2.5-72B	RL 学“formal reasoning pattern”	miniF2F 80.7% (`pass@8192`) [19]
DeepSeek-Prover-V2	2025	671B	子目标分解 + RL	miniF2F-test 88.9%；PutnamBench 49/658 [20]
Goedel-Prover-V2	2026	8B / 32B	合成加难 + 自我纠错 + 检查点平均	8B 84.6%@32；32B 88.1%@32 / 90.4% 自纠错；PutnamBench 86@184 [21]
POETRY	2024	Isabelle	逐层 sketch + `sorry` 延后	miniF2F 平均 +5.1%；最长证明 10→26 步 [14]
Bourbaki	2025	7B 集成	自生成目标条件 MDP + MCTS	PutnamBench 26 题（同规模 SOTA）[29]
ProofAug	2025	`deepseek-math-7b`	细粒度自动化插入	Isabelle 66.0%@2100 查询；Lean `pass@1` 44.3%→50.4% [30]

表 2: 代理式（agentic）系统的代表结果

系统	年	基座/规模	架构要点	原文结果口径
COPRA	2023	GPT-4	状态化回溯搜索 + 引理检索	`pass@1` 超过微调式 ReProver [12]
AxProverBase	2026	Opus 4.5	Proposer/Compiler/Reviewer/Memory	PutnamBench 54.7%@1；FATE-M 98.0%；FATE-H 66.0%；FATE-X 24.0%；LeanCat 59.0% [25]
Numina-Lean-Agent	2026	Claude Code + MCP	单一通用编码 agent + 技能工具	Putnam 2025 12/12；形式化 Brascamp–Lieb [26]
ReCent-Prover	2026	Rocq	反思式验证 + 规划式检索	CoqStoq 相对 +22.58%（同调用预算）[28]
AI4Math-2026 T2	2026	商用模型	6 子代理 + 确定性状态机	34 题解 30（600/1000 分）[24]
Hilbert	2026	商用模型	非形式 LLM + 证明 LLM + 验证器 + 检索	miniF2F 99.2%；PutnamBench 462/660 (70.0%) [22]
Seed Prover 1.5	2026	闭源	NL 证明器 + Sketch + Agentic Prover 递归	IMO 2025 35/42；Putnam 2025 11/12；PutnamBench 88%；FATE-H 80%；FATE-X 33% [23]
OProver	2026	32B	代理式证明轨迹回流训练 + 检索记忆	MiniF2F 93.3%@32；ProverBench 58.2%；PutnamBench 11.3% [27]
DreamProver	2026	–	wake–sleep 引理库演化	成功率提升、证明更短、成本更低 [38]
CircuitProver	2026	–	硬件 Lean 证明 + 可复用库	63/63；朴素 agent 92.1%；证明长度 −16.3%、时间 −23.2% [39]

3.4 为什么这些数字不能直接排队

这是本报告**最重要的方法论结论之一**，直接影响项目自己怎么报指标。

1. **`pass@k` 不同，差几个数量级**。Kimina 的 80.7% 是 `pass@8192` [19]；AxProverBase 的 54.7% 是 `pass@1` 且最多 50 轮迭代 [25]；Goedel-V2 报 `pass@32` 与 `pass@184` [21]。把 `pass@1` 与 `pass@8192` 并列会得出完全错误的强弱判断。
2. **同一基准的不同版本/子集**。 PutnamBench 在文献中出现 658 与 660 两种总数（本报告两处原文分别如此），且 “PutnamBench 88%”（Seed [23]）与 “SeedProver 50.4%”（Hilbert [22]）并存——后者很可能评估的是更早版本或更严格设置。[未核实: Seed 的 88% 与 Hilbert 转述的 50.4% 之间的差异原因，需读两份原文的评测章节才能定论]
3. **时间/成本预算不同**。 Seed Prover 1.5 的 IMO 结果是 16.5 小时内产出前五题；AxProverBase 是 50 轮迭代。同样的通过率，成本可能差两个数量级。
4. **基座模型不同且快速迭代**。 2026 年的代理式结果多用闭源前沿模型（Opus 4.5 等），这类结果

不能迁移到弱模型上；而专用模型结果绑定特定 Lean/Mathlib 版本。

对本项目的直接含义：自建评测必须同时报告 `pass@k`、迭代/采样预算、wall-clock、token 与 Lean/Mathlib 版本哈希，否则既不能与文献比较，也不能与自己的历史版本比较。这与 `AGENT.md` 第 9 节的口径一致，但本报告建议**额外强调 pass@k 的显式声明**。

3.5 开源工程与工具层

下表复核了 `AGENT.md` 第 7 节的仓库，并补充 2026 年新出现的工程。许可证与活跃度为 2026-10-18 通过 GitHub API 实测。

表 3: GitHub 工程现状（2026-10-18 实测）

仓库	Stars	许可证	最近推送	定位
`lean-dojo/LeanCopilot`	1332	MIT	2026-10-13	Lean 内 LLM 副驾驶 [51]
`lean-dojo/LeanDojo`	846	MIT	2026-01-18	数据抽取 + 程序化交互 [11]
`oOo0oOo/lean-lsp-mcp`	530	MIT	2026-09-30	Lean MCP 工具层（goals/diagnostics/检索）
`cameronfreer/lean4-skills`	464	MIT	2026-09-29	Agent 技能包：prove/autoprove/cascade
`ByteDance-Seed/Seed-Prover`	441	Apache-2.0	2026-02-13	Seed Prover 代码与 Putnam 2025 解答 [23]
`project-numina/numina-lean-agent`	280	无	2026-07-08	Claude Code + MCP 的通用形式化代理 [26]
`frenzymath/Archon`	225	Apache-2.0	2026-08-17	plan agent + prover agents，DAG 编排 [61]
`Goedel-LM/Goedel-Prover-V2`	194	无	2025-08-27	Goedel-Prover-V2 模型与数据 [21]
`stanford-centaur/PyPantograph`	150	Apache-2.0	2026-08-30	Lean 4 程序化交互（proof state 控制）
`lean-dojo/LeanDojo-v2`	147	Apache-2.0	2026-08-10	端到端训练/评测框架（依赖 PyTorch）
`lean-dojo/LeanAgent`	87	无	2025-06-13	终身学习式证明框架
`trishullab/copra`	81	无	2026-08-15	COPRA 官方实现 [12]
`Axiomatic-AI/ax-prover-base`	52	**AGPL-3.0**	2026-09-03	最小代理基线（4 组件）[25]
`agenticsnz/unsorry`	42	Apache-2.0	2026-10-07	分布式 swarm：Git 队列 + kernel 闸门 [62]
`frenzymath/LeanSearch-v2`	24	Apache-2.0	2026-05-18	全局前提检索 [34]
`haoxiongliu/ProofAug`	16	无	2025-06-08	ProofAug 官方实现 [30]
`r-irbe/proof-skills`	2	Apache-2.0	2026-10-16	63 个 skill，含多 Agent 角色契约
`Lean-Prover/lean-prover`	1	无	2026-09-10	自主 Lean 4 证明 agent（§3.5）[64]
`twoto1lab/Ax-prover-repro`	1	无	2026-03-11	3-Agent 复现（Orchestrator/Prover/Verifier）

许可证要点。 可直接复用代码的是 MIT / Apache-2.0；**AGPL-3.0** 的 `ax-prover-base` 若被集成进联网 App 可能触发源码开放义务，需先确认；**无许可证**的仓库（Numina-Lean-Agent、Goedel-Prover-V2、COPRA、ProofAug、Lean-Agent、lean-prover、Ax-prover-repro）默认保留所有权利，只能**只读参考**，不能复制代码。注意 `project-numina/numina-lean-agent` 有 280 star 但**无许可证**，这是

一个容易被忽略的合规陷阱。

一个值得注意的小工程。 `Lean-Prover/lean-prover` 只有 1 个 star，但它的形态和本项目的目标（弱模型 + 有限预算 + 端到端）高度相似：任何 OpenAI 兼容端点、逐难度分层的步数/温度配置、MCP server、会话恢复与分支、best-of-N 与 beam search、本地 Mathlib 检索、自动形式化。其自报成绩是在自带 100 题分级基准上用**免费 27B 端点**得 68/100（trivial 20/20、easy 23/30、medium 21/30、hard 4/20），成本 $0 [64]。这个例子说明：**用免费/廉价端点做一条可用的证明回路是现实的**，难点集中在 hard 档。

4 DeepSeek 模型在数学证明领域的特质

4.1 模型谱系：两条线

调研中最重要的一个结构性发现是：**DeepSeek 在数学上有两条几乎不相交的产品线**，二者能力来源完全不同。

线	代表	能力来源
非形式数学线	DeepSeekMath → V3 → R1 → V4-Pro / V4-Flash	网页数学语料 + GRPO/RL 推理
形式证明线	DeepSeek-Prover → V1.5 → V2	大规模合成 Lean 数据 + 专门训练

非形式数学线。 DeepSeekMath 7B 从 DeepSeek-Coder-Base-v1.5 继续预训练 120B 数学相关 token，在 MATH 上无工具无投票达 51.7%，自洽性 64 样本达 60.9%；其提出 **GRPO**（Group Relative Policy Optimization），在降低 PPO 显存开销的同时提升数学推理 [56]。DeepSeek-V3 是 671B 总参数 / 每 token 激活 37B 的 MoE，14.8T 预训练 token，训练仅 2.788M H800 GPU 小时 [57]。DeepSeek-R1 证明**纯 RL** 就能激励出自我反思、验证、动态策略调整等推理模式，无需人工标注推理轨迹，成果发表于 *Nature* 645:633–638 [58]。

形式证明线。 DeepSeek-Prover 用“自然语言题 → 形式语句 → 过滤低质 → 生成证明”的管线造出 800 万条带证明的 Lean 4 语句，微调 DeepSeekMath 7B，miniF2F-test 达 46.3%@64 / 52% 累计，超过当时 GPT-4 的 23.0%@64，并在 FIMO 上解出 5/148（GPT-4 为 0）[13]。

DeepSeek-Prover-V2 [20] 是这条线的当前顶点，也是**与“多 Agent”关系最直接**的一篇：用 DeepSeek-V3 做**递归定理证明管线**来采数据——V3 先把难题拆成子目标序列，已解子目标的证明合成思维链，再与 V3 的逐步推理结合，构成 RL 的冷启动数据。结果是**把非形式推理与形式证明压进同一个模型**：671B 模型在 miniF2F-test 达 88.9%，PutnamBench 49/658，并提出 ProverBench（325 题，含 2024–2025 AIME 15 题，模型解出 6 题，而 DeepSeek-V3 用多数投票解出 8 题）[20]。

该结果的两个含义。

- **形式与非形式数学的差距在缩小**：V3（非形式）解 8/15，Prover-V2（形式）解 6/15，差距已很小 [20]。
- **“分解器用通用模型、证明器用专用模型”是被验证过的分工**。这正是 §6.1 建议的级联原型，而且 DeepSeek 自己的数据管线就是按这个分工造的。

4.2 形式化赛道上 DeepSeek 已不是第一

必须清楚记录：DeepSeek-Prover-V2 的 671B 规模正在被**小一到两个数量级**的模型超越。

- Goedel-Prover-V2-8B 在 `pass@32` 下达 84.6%，**以约 1/80 的规模超过 DeepSeek-Prover-V2-671B**；32B 版达 88.1%@32、自纠错模式 90.4%；PutnamBench 上 86 题 @184，对照 DeepSeek-Prover-V2 的 47 题 @1024，模型小约 20 倍、算力显著更低 [21]。
- 这意味着形式化能力的主要驱动力是**训练数据的构造方式**（合成加难、自我纠错、偏好优化、代理轨迹回流）而不是参数规模。OProver 的报告给出同样信号：32B 在 MiniF2F 达 93.3%@32 [27]。

4.3 “DeepSeek Flash” 到底是什么

项目文档写的是 “DeepSeek Flash API”。调研确认，在当前时点这最可能指 **DeepSeek-V4-Flash**。

- DeepSeek-V4 系列预览版含两个 MoE 模型：**V4-Pro 1.6T（激活 49B）**与 **V4-Flash 284B（激活 13B）**，上下文**均达一百万 token**。架构上有 CSA（压缩稀疏注意力）+ HCA（重压缩注意力）的混合注意力、Manifold-Constrained Hyper-Connections、以及 Muon 优化器；预训练逾 32T token。在百万 token 设置下，V4-Pro 单 token 推理 FLOPs 仅为 V3.2 的 27%、KV cache 为 10% [59]。
- 第三方评测：DeepSeek V4 Flash 在 AIME 2026 上得 **96.7%**（14 个模型中第 2），上下文 1,048,576 token，MIT 许可、权重开放；报告指出的失败模式是复杂组合计数与中间步骤的小数值误差 [60]。
- `Lean-Prover/lean-prover` 的 README 中，免费模型列表明确包含 `deepseek-v4-flash-free` [64]，说明该模型已在第三方聚合平台上以免费额度提供。

4.4 对本项目的四条判断

1. **Flash 的强项是 “非形式数学 + 长上下文 + 便宜”，不是 “写长 Lean 证明”。**96.7% 的 AIME 是单发自然语言题；而形式化需要的是 tactic 级精确语法与库内声明名。目前**不存在** Flash 规模的形式化专用模型；DeepSeek 的形式化能力集中在 671B 的 Prover-V2 上 [20]。
2. **因此应当把 Flash 定位为 “高频短程决策器”：**分解子目标、读诊断写修复建议、判断该不该换策略、以及在长上下文中做跨文件上下文管理。这正是 `AGENT.md` 第 6 节的判断，本报告对该判断给出**支持性证据**：V4 系列的百万上下文使 “把整个 Lean 文件 + 历史错误 + 检索结果一次性放进 prompt” 在工程上可行 [59]。
3. **不要让 Flash 一次生成整段证明。**AxProverBase 的实证是 “迭代优于多次单发”，尤其在样本效率与成本上 [25]；把预算花在**更多轮修复**而非更大采样上，对弱模型更划算。
4. **成本结构上，Flash 反而是优势。**在同样 `pass@1` 目标下，弱模型 + 更多轮次往往比强模型 + 少量轮次便宜；而无训练、纯 API 的方案（§3.5 的 ax-prover / lean-prover 形态）不需要 GPU [25, 64]。

需要实测的三件事。 以下三项本报告**无法**在没有 API key 的情况下确认，应列为 P0 前置验证：

- V4-Flash 的 **工具调用 / JSON 结构化输出**在长上下文下是否稳定；
- 单轮可稳定产出的 Lean 代码长度与 **“一次改对一处” 的修复成功率**；

- 每百万 token 的**实际计费与延迟**，以及是否有缓存折扣。

5 其他关键调研

5.1 基准：饱和、偏置与不可比

表 4: 主要基准及其已知偏置

基准	规模	层次	关键事实 / 偏置
miniF2F	488	奥赛	244 valid / 244 test；跨 Metamath/Lean/Isabelle/HOL Light [3]。**已饱和**：99.2% [22]，区分度接近耗尽
PutnamBench	658/660	本科	大学数学；文献总数口径不一致，见 §3.4
ProofNet	371	本科	Lean 3 语句 + 自然语言语句 + 自然语言证明 [48]；现为自动形式化主基准
FIMO	149	IMO	IMO 短名单题；早期 GPT-4 解 0 题 [49]
FormalMATH	5560	高中–本科	最强调 “实际采样预算”：最强模型仅 **16.46%**；存在明显领域偏置（代数强、微积分弱）[43]。并发现**自然语言解题提示反而降低**形式化证明成功率
TaoBench	–	本科	用 Tao《Analysis I》的自建定义体系，与 Mathlib 等价版本配对；SOTA **平均下降约 26%**，说明瓶颈是**跨定义框架泛化**而非题目难度 [44]
MathAtlas	∼52k	研究生	103 本研究生教材；语句正确率最高 9.8%、定义 16.7%；最深依赖子集 MA-Hard 上仅 **2.6%** [45]
TheoremBench	∼100	经典定理	区分 “主定理” 与 “自动抽取的支撑子定理”；发现现有证明器**偏好简单子定理**，且靠长而低效的 tactic 轨迹而非紧凑证明计划解题 [46]
NTP4VC	–	真实代码	首个面向验证条件（VC）的多语言真实基准（Linux/Contiki 等），VC 证明仍是瓶颈 [47]
ProverBench	325	混合	DeepSeek 提出，含 AIME 24–25 的 15 题 [20]
LeanCat	–	–	AxProverBase 使用，59.0% 对 Gemini 3 Pro 14.0% [25]
FATE-M/H/X	–	代数/本科/博士	分层难度；M 与 H/X 差距极大（98.0% vs 66.0% vs 24.0%）[25]

结论。 项目选 miniF2F 作为首个评测集是**合理的起步选择**（社区标准、跨系统、工具链成熟），但必须清楚两点：(a) 它在 2026 年已接近饱和，**不能用来区分强系统**；(b) 应当从第一天起就预留 **PutnamBench 子集 + FATE-H/X 风格分层**作为升级基准，否则会出现 “miniF2F 涨点但真实能力没变” 的假进展。

5.2 反馈信号：什么在真正起作用

跨系统的证据指向同一个结论：**反馈的粒度比模型规模更重要**。

- **结构化 goal state 是关键增量。**AxProverBase 明确把 “在 `sorry` 处抽取 goal state” 作为 Compiler 组件的核心设计 [25]；Lean 生态的 `lean-lsp-mcp` 与 PyPantograph 提供了 goals/diagnostics 的机器可读接口 [63]。
- **评审角色能挡掉作弊但不提升搜索。**AxProverBase 的 Reviewer 负责 “定理语句未被篡改、无 `sorry`、无作弊 tactic”，这属于**安全性**而非能力 [25]。值得项目抄的是：把 “是否作弊” 与 “是否证出” 分成两个独立判定。
- **critic 的价值在于偏好而非判定。**InternLM2.5-StepProver 的 critic 提供运行时偏好引导搜索，把 59.4% 提到 65.9% [17]。
- **“反思” 需要被显式设计。**ReCent-Prover 的 “验证时带反思” 先审视自己生成的 tactic、识别潜在误用并写失败摘要，再进入下一轮 [28]。这是可以直接用 prompt 实现的低成本改进。
- **记忆要压缩成可读条目。**AxProverBase 的 Memory 把失败教训总结成 “实验记录本” 以防重复犯错 [25]；直接塞历史轨迹会迅速耗尽上下文。

5.3 检索与前提选择

- **检索质量确实传递到证明成功率。**LeanSearch v2 在固定 prover 回路的对照实验中：20%（LeanSearch v2）vs 16%（次优）vs 4%（无检索）[34]。这是本文档中**最干净的一组消融**。
- **从 “找声明” 升级到 “找整套前提”。**LeanSearch v2 的 global premise retrieval 与 DRIFT 的 “先分解再检索” 都基于同一观察：非形式语句与 Mathlib 定理之间没有直接映射，必须先拆子组件才能检索到正确前提 [34, 40]。
- **前提选择已经成为独立子领域，**本地文献库编号 101–122 的 22 篇全部属于此（LeanSearch v2、Lean Finder、LeanExplore、LeanPremise、REAL-Prover、DreamProver、DRIFT、MathlibLemma、Semantic Search over 9M theorems 等）。其中 MathlibLemma 报告用 LLM 管线产出 1506 条 Lean 校验过的 “folklore” 引理，并把 4028 条非平凡语句做成基准 [41]。
- **库的结构本身值得研究。**Mathlib 的依赖网络分析给出 308,129 个声明、840 万条边、7,563 个模块；人类分类与逻辑结构有 50.9% 的耦合偏差；开发者平均只用到导入作用域的 1.6% [42]。这解释了为什么 “全量 Mathlib 检索” 效果差、而 “精准前提检索” 效果好的结构性原因。
- **对项目的含义：**如果第一版不做检索，就等于放弃了文献中最确定的增益项（+16 个百分点绝对值）。但检索会引入一个重依赖（索引 + 服务），需要权衡。折中方案是先接 **LeanSearch 的公开 API**（`leansearch.net`）[34] 或 MCP，而不是自建索引。

5.4 验证协议与作弊面

必须审计的五类绕过手段。

1. `sorry` / `admit` / `by_contra!` 类的**未完成占位**；
2. **新增公理**（`axiom`）或引入 `Classical.choice` 之外的非常规公理；
3. **修改定理语句**（把命题改弱、加隐含前提、改量词）；
4. **`native_decide`** 等把可信基扩展到编译器/运行时的 tactic；
5. **把目标搬进更容易的类型**（例如借助 `False` 消解、或利用不一致的本地上下文）。

已有的可抄做法。

- **定理语句逐字比对**：AxProverBase 的 Reviewer 显式检查 statement preservation [25]。
- **公理审计**：FTAP 的形式化把 “每个主定理的公理被 build 强制钉死到 Mathlib 的经典默认集” 做成了构建期闸门，并且全库 `sorry`-free、toolchain 固定可复现 [50]。这是目前见到的**最严格的工程化验证协议**，值得直接借用其形式（build-enforced axiom gate）。
- **kernel 作为唯一闸门**：`unsorry` 的设计原则是 “仓库是队列、kernel 是裁判”，CI 的 Gate A 复验后才自动合并 [62]。
- **证明绕过筛查**：MathlibLemma 对 1506 条 Lean 校验证明额外做 “proof-bypass screen” [41]。

5.5 成本、时间与 “迭代 vs 采样”

- **迭代优于多次单发**：AxProverBase 明确报告迭代式相对多次单发在样本效率与成本上的一致优势 [25]。
- **查询预算的差距是数量级的**：ProofAug 用每题 2100 次查询达到 66.0%，而此前 SOTA 用 16384 次查询才 56.1% [30]。这说明**单位查询的产出**可以差一个数量级——报告 `pass@k` 而不报查询数是没有意义的。
- **搜索复杂度并非必需**：BFS-Prover 用相对简单的 best-first 就在 7B 上达到 72.95%，挑战了 “必须 MCTS/价值网络” 的成见 [18]。对算力有限的项目，这是好消息。
- **零成本端点是现实的**：lean-prover 用免费 27B 端点跑出 68/100、成本 $0 [64]。

5.6 工程基础设施：MCP 已成为事实标准

- **MCP（Model Context Protocol）**在 2025–2026 年成为 Lean 工具接入 LLM 的统一方式。Numina-Lean-Agent 明确把 “MCP 使工具扩展与自主调用成为可能、避免复杂设计” 作为其范式选择的理由之一 [26]；LeanExplore 直接提供 MCP 集成 [36]；`lean-lsp-mcp` 提供 goals/diagnostics/LeanSearch/Loogle/Lean Finder/Lean Hammer 的工具层。
- **程序化 proof state 控制**：PyPantograph 提供 tactic 执行、proof state 管理与整文件检查 [63]；需要比 “整文件编译” 更细的控制时这是必要条件。
- **端到端评测框架**：LeanDojo-v2 覆盖训练/评测/部署，但依赖 PyTorch/CUDA，偏重 [11]。
- **重要的反面证据**：Numina-Lean-Agent 的主张是**直接用一个通用编码 agent 当形式数学推理器**，理由是 “换更强的基座模型即可提升，无需训练”，并用 Claude Code + Numina-Lean-MCP 在 Putnam 2025 上 12/12 [26]。这与 “必须自研多 Agent 编排” 的直觉相反。

5.7 一个必须写清楚的张力：多 Agent 到底带来了什么

把上文的证据合起来看，会得到一个与项目原计划**部分冲突**的结论：

观察：2026 年最强的两个公开结果分别来自两条相反的路：

- **多组件流水线**：Hilbert（4 组件 + 递归分解）99.2% / 70.0% [22]；Seed Prover 1.5（NL 证明器 + Sketch + Agentic Prover 递归）IMO 35/42 [23]；AI4Math T2（6 子代理 + 确定性状态机）[24]。
- **单一通用代理**：Numina-Lean-Agent（Claude Code + MCP）Putnam 2025 12/12 [26]。

解读：多 Agent 的收益主要来自**上下文隔离**与**确定性控制**，而不是“角色数量”。当基座模型足够强、且工具接口足够好时，单代理 + 好工具可以达到同等水平；当基座模型较弱（如本项目第一版要用的 Flash）时，**上下文隔离的价值上升**，因为弱模型最容易被长上下文里的无关信息带偏。

这个解读对项目是**有利**的：它说明“弱模型 + 严格编排”是有理论依据的路线，而不是无奈之举；但同时警告：**如果角色划分不能带来上下文隔离或可测量的贡献，它就只是开销**。这也正是 `AGENT.md` 第 5.4 节“避免角色扮演式多 Agent”的量化依据。

6 对本项目的启示与建议

6.1 级联原型的文献依据

`AGENT.md` 第 5 节的 L0–L4 级联与文献中的分工高度一致，且每个层级都有可直接引用的证据：

层	建议实现	文献依据
L0	tactic cascade	lean4-skills 的 `rfl` → `simp` → `ring` → `linarith` → `nlinarith` → `omega` → `exact?` → `apply?` → `grind` → `aesop` 顺序；Lean Copilot 报告自动完成 74.2% 步数（aesop 40.1%）[51]
L1	Prover ↔ Lean 反馈	AxProverBase 的 Proposer/Compiler 回路 [25]；Lean-STaR 的“先思考再给 tactic” [16]
L2	检索 + Critic	LeanSearch v2 的 20% vs 4% 消融 [34]；ReCent-Prover 的反思式验证 [28]；InternLM critic +5.5 点 [17]
L3	Planner + 递归分解	Seed Prover 1.5 的 Sketch + 递归 [23]；Hilbert 的递归子目标 [22]；POETRY 的 `sorry` 延后 [14]
L4	多候选 / 升级模型	Goedel-V2 的自纠错模式 88.1%→90.4% [21]；Bourbaki 的 7B 集成 [29]

6.2 一个必须纠正的预期

`AGENT.md` 摘要写“DeepSeek Flash 数学能力有限，Agent 结构必须补足知识与搜索能力”。调研结论是：**这个判断部分过时**。V4-Flash 在 AIME 2026 上 96.7% [60]，单发非形式数学能力并不弱；弱的是**形式化**（精确语法、库内声明名、类型错误定位）与**长程一致性**。因此“Agent 结构”要补的不是“数学知识”，而是：

1. **形式语法正确性**（靠编译器反馈闭环，而不是靠模型记性）；
2. **库内定位**（靠检索，而不是靠模型记忆 Mathlib 声明名）；
3. **长程一致性**（靠子目标分解 + 逐步验证，而不是靠一次生成）。

这个区别会直接改变 prompt 设计：不要指望模型“想起”引理名，而要让它**先检索再引用**。

6.3 五个可做的差异化方向

在“已有工作已经很强”的前提下，项目需要找到不被碾压的切入点。以下是调研中浮现的、**仍显空白且与弱模型策略相容**的方向：

1. **弱模型下的成本–准确率 Pareto 曲线**。现有工作几乎都在追最高通过率，很少系统报告“同等预算下弱模型 + 编排”的曲线。lean-prover 的 \$0 / 68 分 [64] 与 ProofAug 的查询效率对比 [30] 说明这个维度有真实空间。
2. **上下文预算的算法**。多 Agent 的核心价值是上下文隔离（§6.1），但“如何决定把哪一段 Lean 文件、哪些历史错误、哪几个检索结果放进当前角色的 prompt”目前都是工程 trick，没有公开的算法化处理。
3. **失败诊断的分类学与可操作化**。ReCent-Prover 的失败摘要 [28]、AxProverBase 的实验记录本 [25] 都是萌芽。把 Lean 错误系统性地归到“缺引理 / 类型不匹配 / 需要归纳 / 需要换策略 / 语句理解错”并度量每类的修复率，是有价值的实证工作。
4. **引理库的跨题迁移是否真实有效**。LEGO-Prover 报告库带来 +3.3 点 [15]、DreamProver 用 wake–sleep 演化库 [38]、CircuitProver 报告库减少 16.3% 证明长度 [39]。但这些都是在**强基线**上做的。在弱模型上，库的收益/噪音比是否更有利，尚无公开答案。这也正好衔接项目此前“引理库/前提选择”的积累（`arxiv` 论文编号 101–122）。
5. **中文/教学场景的形式化**。现有 benchmark 与系统全部面向英文与竞赛题。若项目的 App 目标包含中文用户，这是一个无人竞争的方向（但需注意 ProofNet/FormalMATH 的领域偏置 [48, 43]）。

6.4 建议的最小可行路线（P0–P1）

基于上述，建议把 `AGENT.md` 第 12 节的计划**收紧**为：

1. **环境**：新建 Lean 工程，`lean-toolchain` 锁 `leanprover/lean4:v4.21.0`，直接复用本机已编译的 Mathlib（2687 个 olean，2.31 GB）——这一步在本机已具备条件，无需重新下载编译。
2. **单题回路**：实现“生成 → `lake env lean` → 解析诊断 → 修复”，先不引入任何 LLM 角色划分，只做一条 loop。目标：10 道 miniF2F 上跑通端到端并产出 JSONL。
3. **基线**：加 L0 tactic cascade，与“裸 LLM loop”对比，量化 cascade 的贡献。
4. **反馈升级**：把诊断从“stderr 文本”换成 **结构化 goal state**（接 `lean-lsp-mcp` 或 PyPantograph）[63]，度量增益。
5. **检索**：在固定回路下接 LeanSearch 公开 API，复现 LeanSearch v2 报告的那组“有/无检索”消融 [34]。
6. **再谈多 Agent**：只有在第 4、5 步证明“上下文隔离”带来可测量收益后，才引入 Planner/Critic 角色。

这个顺序的关键是**先证明每一层有效，再增加角色**——与 §6.1 的张力结论一致。

6.5 待验证清单

以下事项本报告无法确认，必须由项目自己实测（与 `AGENT.md` 第 11 节“尚未决定的问题”合并）：

1. V4-Flash 的工具调用、JSON 输出、长上下文稳定性与计费（§4.4）。
2. 本机 Mathlib v4.21.0 与目标 Lean 工程的兼容性，以及是否需要重建缓存。
3. miniF2F 的 Lean 4 版本在本地能否逐题编译通过（历史上有语句随 Mathlib 演进而失效的问题）。
4. 中文 LaTeX / Lean 混排环境下诊断信息的编码问题（Windows 控制台编码曾导致本报告的数据抽取失败，见附录 A）。
5. PutnamBench 的准确总数与版本（658 或 660，§3.4）。

7 结论

1. **架构上，2026 年的主流是“确定性编排 + 受限 LLM 角色”**，不是自由对话式多 Agent。角色划分的价值来自上下文隔离与可测量贡献，而不是角色数量；Hilbert、Seed Prover 1.5、AI4Math T2 与 Numina-Lean-Agent 分别从两个方向证实了这一点。
2. **结果上，miniF2F 已经饱和**（99.2%），真正区分系统的是 PutnamBench、FormalMATH、TaoBench、FATE-X 这类**分布外 / 高依赖深度**的基准；而且文献中的 `pass@k` 与预算差异巨大，任何跨系统比较都必须先对齐口径。
3. **DeepSeek 的两条线要分开看**：非形式数学（V4-Flash，AIME 2026 96.7%）已经很强，形式化（Prover-V2 671B）强但已被小模型超越。项目用 Flash 是合理的，但要把它的角色定义为**规划者/批评者/修复者**，而不是 **Lean 代码生成者**。
4. **最确定的两个增益项是“结构化反馈”和“检索”**，分别有 AxProverBase 与 LeanSearch v2 的干净消融支撑；最不确定的是“引理库/记忆”，它在强基线上有效，在弱基线上待验证——而这恰好是项目可能做出差异的地方。
5. **下一件该做的事不是搭框架**，而是把“10 道 miniF2F 上的端到端回路 + JSONL 日志”跑通，用真实数字替换本报告中的所有引用数字。

A 数据来源与可复现性

- **本地文献**：仓库 `arxiv 论文/` 下 `harvest_all.json`（1225 条 arXiv 记录）提供 100 篇的题录与摘要。
- **中间产物**：本目录下 `abstracts.txt`（95 条关键论文摘要）、`abstracts.json`（结构化版本）、`missing_pdf_text.json`（9 篇不在题录中的论文的首页文本：DreamProver、REAL-Prover、DRIFT、Premise Selection for a Lean Hammer、Semantic Search over 9M Theorems、Machine-Learned Premise Selection for Lean、Graph Sequence Learning、Millions of Lemmas、Textual+Structural Premise Selection）。
- **外部来源**：arXiv 官方 `abs` 页面（DeepSeekMath、DeepSeek-R1、DeepSeek-V3、DeepSeek-V4、Kimina-Prover、AxProverBase）；ICLR 2026 论文集页面（Hilbert、Goedel-Prover-V2）；ICML 2026 workshop 页面（AI4Math Track 2）；ByteDance Seed 官方博客（Seed Prover 1.5）；LayerLens 第三方评测（DeepSeek V4 Flash on AIME 2026）。
- **GitHub**：通过 `api.github.com` 读取仓库元数据（stars / license / pushed_at / archived /

description），通过 `github.com/<repo>/raw/<branch>/README.md` 读取 README。**注意**：本环境的 `web_fetch` 对 `github.com` 与 `raw.githubusercontent.com` 解析到非公网地址而失败，因此全部 GitHub 数据由 PowerShell 直连取得；`api.github.com` 的匿名配额为 60 次/小时，查询 24 个仓库后耗尽，故 README 改用网页 raw 路径。

- **一个已修复的环境问题**：会话开始时工作区 `D:\bianma\code\大创 2` 因缺少 `WRITE_OWNER` 导致所有 shell 命令失败；已通过权限诊断脚本为当前用户补齐完全控制（文件内容与所有者未改动，备份与回滚脚本位于 `D:\bianma\code\dsh-acl-recovery`）。另：Windows 控制台默认 GBK 编码曾导致 Python 输出中文/特殊符号时抛 `UnicodeEncodeError`，故所有抽取脚本改用 `-X utf8` 与 `sys.stdout.reconfigure(encoding='utf-8')`。

B 未核实与存在冲突的数字

1. **PutnamBench 总数**：Hilbert 写 660，DeepSeek-Prover-V2 写 658。本报告两处均按原文引用，未做统一。
2. **SeedProver 在 PutnamBench 上的成绩**：Seed 官方称 1.5 版解出 “full PutnamBench 的 88%” [23]；Hilbert 论文称 “SeedProver 50.4%” [22]。两者**很可能评估对象不同**（不同版本或不同预算），本报告无法从摘要层判定，故并列呈现。
3. **DeepSeek-V4-Flash 的 “输出冗长”**：该说法来自第三方网关博客标题（*Cheap, Verbose, Matches V4 Pro at Math*），正文未能取得，[未核实: 冗长程度与 token 效率]。
4. **V4-Flash 在形式化任务上的表现**：目前**没有任何**公开评测把 V4-Flash 放到 Lean 证明任务上。本报告 §4.4 的四条判断是基于 “AIME 96.7% + 无形式化专用训练” 的推断，[未核实: 实际 Lean 通过率]。
5. **Goedel-Prover-V2 的 PutnamBench 换算是本报告做的**：86 题 @184 换算为 “13.1%” 是为与 AxProverBase 的百分比对照，原文只给题数。
6. **Agent 各角色的边际贡献**：除 InternLM2.5-StepProver 的 critic（+5.5 点）[17] 与 LeanSearch v2 的检索（+16 点绝对值 / 4%→20%）[34] 外，其余系统的角色级消融多未在摘要层给出，需读全文。
7. **本报告未覆盖**：AlphaProof（Nature 2025）与 Aristotle（Harmonic）等闭源系统的细节，仅通过二手材料知晓其存在与量级；自动形式化方向（本地文献 001–010、079–085）仅在与证明闭环相关处提及，未做系统展开。

参考文献

[1] L. de Moura, S. Ullrich. The Lean 4 Theorem Prover and Programming Language. CADE-28, 2021. DOI:10.1007/978-3-030-79876-5_37.

[2] The mathlib Community. The Lean mathematical library. CPP 2020. arXiv:1910.09336.

[3] K. Zheng, J. M. Han, S. Polu. MiniF2F: a cross-system benchmark for formal Olympiad-level mathematics. arXiv:2109.00110.

[4] S. Polu, I. Sutskever. Generative Language Modeling for Automated Theorem Proving. arXiv:2009.03393.

[5] J. M. Han, J. Rute, Y. Wu, E. W. Ayers, S. Polu. Proof Artifact Co-training for Theorem Proving with Language Models. arXiv:2102.06203.

[6] G. Lample, M.-A. Lachaux, T. Lavril, et al. HyperTree Proof Search for Neural Theorem Proving. arXiv:2205.11491.

[7] A. Q. Jiang, W. Li, S. Tworkowski, et al. Thor: Wielding Hammers to Integrate Language Models and Automated Theorem Provers. arXiv:2205.10893.

[8] M. Mikuła, S. Tworkowski, S. Antoniak, et al. Magnushammer: A Transformer-Based Approach to Premise Selection. arXiv:2303.04488.

[9] E. First, M. N. Rabe, T. Ringer, Y. Brun. Baldur: Whole-Proof Generation and Repair with Large Language Models. arXiv:2303.04910.

[10] A. Q. Jiang, S. Welleck, J. P. Zhou, et al. Draft, Sketch, and Prove: Guiding Formal Theorem Provers with Informal Proofs. arXiv:2210.12283.

[11] K. Yang, A. M. Swope, A. Gu, et al. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. arXiv:2306.15626.

[12] A. Thakur, G. Tsoukalas, Y. Wen, J. Xin, S. Chaudhuri. An In-Context Learning Agent for Formal Theorem-Proving (COPRA). arXiv:2310.04353.

[13] H. Xin, D. Guo, Z. Shao, et al. DeepSeek-Prover: Advancing Theorem Proving in LLMs through Large-Scale Synthetic Data. arXiv:2405.14333.

[14] H. Wang, H. Xin, Z. Liu, et al. Proving Theorems Recursively (POETRY). arXiv:2405.14414.

[15] H. Wang, H. Xin, C. Zheng, et al. LEGO-Prover: Neural Theorem Proving with Growing Libraries. arXiv:2310.00656.

[16] H. Lin, Z. Sun, S. Welleck, Y. Yang. Lean-STaR: Learning to Interleave Thinking and Proving. arXiv:2407.10040.

[17] Z. Wu, S. Huang, Z. Zhou, et al. InternLM2.5-StepProver: Advancing Automated Theorem Proving via Critic-Guided Search. arXiv:2410.15700.

[18] R. Xin, C. Xi, J. Yang, et al. BFS-Prover: Scalable Best-First Tree Search for LLM-based Automatic Theorem Proving. arXiv:2502.03438.

[19] H. Wang, M. Unsal, X. Lin, et al. Kimina-Prover Preview: Towards Large Formal Reasoning Models with Reinforcement Learning. arXiv:2504.11354.

[20] Z. Z. Ren, Z. Shao, J. Song, et al. DeepSeek-Prover-V2: Advancing Formal Mathematical Reasoning via Reinforcement Learning for Subgoal Decomposition. arXiv:2504.21801.

[21] Y. Lin, S. Tang, B. Lyu, et al. Goedel-Prover-V2: Scaling Formal Theorem Proving with Scaffolded Data Synthesis and Self-Correction. ICLR 2026.

[22] S. Varambally, T. Voice, Y. Sun, Z. Chen, R. Yu, K. Ye. Hilbert: Recursively Building Formal Proofs with Informal Reasoning. ICLR 2026.

[23] ByteDance Seed. Seed Prover 1.5: Advanced Mathematical Reasoning through a Novel Agentic Architecture. 2025-12-24. Tech report arXiv:2512.17260.

[24] B.-H. Hwang, J. La, C.-h. Lee, H. Lim. Technical Report for AI4Math-2026 Track 2: Mechanically Orchestrated LLM Sub-Agents for Theorem Proving in Lean 4. ICML 2026 Workshop (3rd AI for Math).

[25] B. Requena, A. Letson, K. Nowakowski, I. Beltran-Ferreiro, L. Sarra. A Minimal Agent for Automated Theorem Proving. ICML 2026. arXiv:2602.24273.

[26] J. Liu, Z. Zhou, Z. Zhu, et al. Numina-Lean-Agent: An Open and General Agentic Reasoning System for Formal Mathematics. arXiv:2601.14027.

[27] D. Ma, K. Ma, S. Guo, et al. OProver: A Unified Framework for Agentic Formal Theorem Proving. arXiv:2605.17283.

[28] Y. Sun, C. Shi, H. Lyu, Y. Xiong. On Reasoning-Centric LLM-based Automated Theorem Proving (ReCent-Prover). arXiv:2604.19558.

[29] M. Zimmer, X. Ji, R. Tutunov, et al. Bourbaki: Self-Generated and Goal-Conditioned MDPs for Theorem Proving. arXiv:2507.02726.

[30] H. Liu, J. Sun, Z. Li, A. C. Yao. ProofAug: Efficient Neural Theorem Proving via Fine-grained Proof Structure Analysis. ICML 2025. arXiv:2501.18310.

[31] Y. Qian, J. Clune, C. Barrett, J. Avigad. Lean-auto: An Interface between Lean 4 and Automated Theorem Provers. arXiv:2505.14929.

[32] A. Mohamed, T. Mascarenhas, H. Khan, et al. Lean-SMT: An SMT tactic for discharging proof goals in Lean. arXiv:2505.15796.

[33] T. Zhu, J. Clune, J. Avigad, A. Q. Jiang, S. Welleck. Premise Selection for a Lean Hammer. ICLR 2026. arXiv:2506.07477.

[34] G. Gao, Z. Sun, J. Jiang, et al. LeanSearch v2: Global Premise Retrieval for Lean 4 Theorem Proving. arXiv:2605.13137.

[35] J. Lu, K. Emond, K. Yang, et al. Lean Finder: Semantic Search for Mathlib That Understands User Intents. arXiv:2510.15940.

[36] J. Asher. LeanExplore: A search engine for Lean 4 declarations. arXiv:2506.11085.

[37] Z. Shen, N. Huang, F. Yang, et al. REAL-Prover: Retrieval Augmented Lean Prover for Mathematical Reasoning. arXiv:2505.20613.

[38] Y. Zhang, J. Sun, H. Bi, et al. DreamProver: Evolving Transferable Lemma Libraries via a Wake-Sleep Theorem-Proving Agent. arXiv:2604.26311.

[39] Z. Yang, W. Fang, C. Chen, Z. Xie, H. Zhang. CircuitProver: Agentic Lean 4 Theorem Proving with Reusable Circuit Proof Library for Hardware Verification. arXiv:2607.27259.

[40] M. Zhang, P. Borchert, M. Gritta, G. Lampouras. DRIFT: Decompose, Retrieve, Illustrate, then Formalize Theorems. ICLR 2026. arXiv:2510.10815.

[41] X. Liu, Z. Xie, A. Moeini, et al. MathlibLemma: Folklore Lemma Generation and Benchmark for Formal Mathematics. arXiv:2602.02561.

[42] X. Li, N. Peng, S. Severini, P. Shafto. The Network Structure of Mathlib. arXiv:2604.24797.

[43] Z. Yu, R. Peng, K. Ding, et al. FormalMATH: Benchmarking Formal Mathematical Reasoning of Large Language Models. arXiv:2505.02735.

[44] A. K. Taylor, J. Zhang, E. Ji, et al. TaoBench: Do Automated Theorem Prover LLMs Generalize Beyond MathLib? arXiv:2603.12744.

[45] N. Patel, N. Arias, D. Babayan, et al. MathAtlas: A Benchmark for Autoformalization in the Wild. arXiv:2605.14061.

[46] Q. Pham, E. Karimov, A. Galichin, I. Oseledets. TheoremBench: Evaluating LLMs on Theorem Proving in Formal Mathematics. arXiv:2606.09450.

[47] Q. Xu, X. Luan, R. Wang, et al. Neural Theorem Proving for Verification Conditions: A Real-World Benchmark. arXiv:2601.18944.

[48] Z. Azerbayev, B. Piotrowski, H. Schoelkopf, et al. ProofNet: Autoformalizing and Formally Proving Undergraduate-Level Mathematics. arXiv:2302.12433.

[49] C. Liu, J. Shen, H. Xin, et al. FIMO: A Challenge Formal Dataset for Automated Theorem Proving. arXiv:2309.04295.

[50] R. Coelho. The Fundamental Theorem of Asset Pricing, Formalized in Lean 4. arXiv:2606.28990.

[51] P. Song, K. Yang, A. Anandkumar. Lean Copilot: Large Language Models as Copilots for Theorem Proving in Lean. arXiv:2404.12534.

[52] F. Bártek, A. Bhayat, R. Coutelier, et al. The Vampire Diary. arXiv:2506.03030.

[53] Z. A. Goertzel, J. Jakubův, C. Kaliszyk, et al. The Isabelle ENIGMA. arXiv:2205.01981.

[54] Z. Li, J. Sun, L. Murphy, et al. A Survey on Deep Learning for Theorem Proving. arXiv:2404.09939.

[55] K. Weng, L. Du, S. Li, et al. Autoformalization in the Era of Large Language Models: A Survey. arXiv:2505.23486.

[56] Z. Shao, P. Wang, Q. Zhu, et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300.

[57] DeepSeek-AI. DeepSeek-V3 Technical Report. arXiv:2412.19437.

[58] DeepSeek-AI. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948. *Nature* 645:633–638 (2025).

[59] DeepSeek-AI. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. arXiv:2606.19348.

[60] LayerLens. DeepSeek V4 Flash on AIME 2026: 96.7% accuracy. 2026-05-13. https://layerlens.ai/blog/deepseek-v4-flash-aime2026

[61] Frenzymath. Archon: AI-assisted Lean project automation with DAG blueprints and multi-agent proving workflows. https://github.com/frenzymath/Archon

[62] Agentics NZ. unsorry: a distributed swarm of autonomous agents proving Lean 4 theorems. https://github.com/agenticsnz/unsorry

[63] Stanford CENTAUR. PyPantograph: A Machine-to-Machine Interaction System for Lean 4. https://github.com/stanford-centaur/PyPantograph

[64] Lean-Prover. lean-prover: an autonomous Lean 4 theorem-proving agent. https://github.com/Lean-Prover/lean-prover